

Secure Cloud for Agent-Generated Small Software

An Intuitive Guide to Model 1 and Model 2

What was combined from the two documents

The attached draft explains the central idea clearly: Model 1 examines source code and dependencies to estimate vulnerability risk, while Model 2 studies runtime behavior to infer the permissions the application actually needs. The earlier APA paper provides the stronger research evidence, dataset plan, model architecture, evaluation logic, and source attribution. This guide combines the simple two-question explanation from the attached draft with the research-backed details from the APA paper.

Scope note: The supplied attached PDF is a one-page browser capture. It shows the overview, the two-model flow, and the beginning of Model 1; only that visible content could be compared. The complete earlier APA paper was used for the detailed model and literature synthesis.

1. What are we building?

We are building a cloud platform that accepts a small application created by an AI coding agent, checks whether it is safe enough to deploy, decides what access it genuinely needs, and then runs it inside an isolated environment. The user should not need to understand vulnerability scanners, Linux permissions, containers, or cloud IAM policies. The platform converts those technical checks into understandable decisions such as allow, review, block, and approve these permissions.

The complete project still contains four required models:

- Model 1: Code and dependency risk classifier.
- Model 2: Behavior-based permission inference model.
- Model 3: Runtime anomaly detector over system-call traces.
- Model 4: Learned cold-start and resource scheduler.

This guide focuses on Models 1 and 2. Model 2 uses successful normal behavior to recommend permissions; it does not replace Model 3, which watches a deployed application for abnormal behavior or attacks.

The two questions in plain language

- Model 1 asks: Is this generated application risky enough that deployment should be blocked or reviewed?
- Model 2 asks: If the application is allowed to run, what is the smallest permission set that still lets it perform its legitimate task?

A worked example used throughout this guide

Imagine that an AI agent creates a receipt-expense application. A user uploads a receipt, the application sends the image to an OCR service, stores the extracted amount, and generates a monthly CSV report. The application appears simple, but generated code could include an unsafe command, a vulnerable package, an exposed secret, unnecessary file access, or permission to contact any website. Models 1 and 2 reduce these risks before the application is shared.

End-to-end flow

1. The user submits the repository.
2. The platform builds an SBOM, scans the code and packages, and runs Model 1.
3. A high-risk app is blocked or sent for review; an acceptable app continues in a controlled sandbox.
4. Functional tests exercise normal behavior while the platform records resource access.
5. Model 2 recommends the minimum policy and explains each requested permission.
6. The user approves sensitive grants; the platform deploys, monitors, schedules, and can roll back the app.

2. Model 1: Code and Dependency Risk Classifier

Purpose

Model 1 is the platform's pre-deployment security gate. It combines what the code appears to do with what is known about its dependencies. It should not produce only a yes/no answer; it should provide a calibrated risk score, the likely problem category, supporting evidence, and a deployment recommendation.

What goes into Model 1

- Source code: functions, imports, API calls, and security-sensitive operations.
- Call and data flow: whether user-controlled data can reach a command, file, query, or network sink.
- Dependencies and SBOM: package names, versions, dependency relationships, and transitive packages.
- Known vulnerability evidence: OSV/CVE matches, severity, affected versions, and available fixes.
- Scanner evidence: secret detection, Semgrep/CodeQL findings, insecure configuration, and suspicious commands.
- Requested permissions: unusually broad file, network, secret, database, or process access.

What the model should return

- Risk score, such as 0.82 on a 0-to-1 scale.
- Risk class: low, medium, or high.
- Likely categories, such as vulnerable dependency, command injection, exposed secret, or unsafe configuration.
- Evidence locations: package version, file, function, line range, or data-flow path.
- Action: allow, require review, or block.

How Model 1 works

1. Create an SBOM and query OSV for known vulnerable package versions.
2. Build a call/data-flow representation and keep security-relevant slices around sensitive operations, adapting the context-reduction idea from LLMxCPG (Lekssays et al., 2025).
3. Encode each code slice with GraphCodeBERT so that tokens and variable data flow are represented together (Guo et al., 2021).
4. Represent functions, dependencies, package versions, and vulnerability relationships as a heterogeneous graph.

5. Fuse code embeddings, graph embeddings, and scanner features; use multi-task heads to predict vulnerability, category, and calibrated risk.
6. Apply explicit deployment rules. For example, a confirmed secret or critical known dependency may trigger review even when the learned risk score is lower.

Illustrative result for the receipt app

Assume the scanner finds that uploaded filenames reach a shell-command function and OSV reports that one package version is affected by a known vulnerability. Model 1 could return: risk = 0.82 (high); evidence = unsafe input-to-command path plus vulnerable dependency; decision = block until the command is removed and the package is updated. This example is illustrative; 0.82 is not a reported project result.

Simplified pseudocode

```
code_slice = security_slice(source_code, sensitive_sinks)
code_vector = graphcodebert(code_slice)
dependency_graph = build_dependency_graph(sbom, osv_matches)
dependency_vector = dependency_gnn(dependency_graph)

features = fuse(code_vector, dependency_vector, scanner_features)
risk, category = risk_model(features)
decision = apply_policy(risk, confirmed_secrets, critical_cves)
```

The pseudocode shows the main idea, not a finished implementation. Rules handle facts that can be verified directly, while ML learns contextual combinations that are difficult to express as fixed rules.

Data and evaluation for Model 1

- Use SecBench.js for vetted server-side JavaScript vulnerabilities, executable payloads, and success oracles (Bhuiyan et al., 2023).
- Use OSV to label vulnerable package versions from the SBOM.
- Use MicroAppSec-100 as the team's original end-to-end benchmark, not as a downloadable dataset.
- Split by repository or complete app family, keep related variants together, and remove near duplicates, following PrimeVul's warning about data leakage (Ding et al., 2025).
- Report precision, recall, F1, PR-AUC, calibration, deployment success, and exploit-blocking recall.

3. Model 2: Behavior-Based Permission Inference

Purpose

Model 2 recommends the least privilege needed for the application to complete its intended work. It combines what the code could access with what successful functional runs actually accessed. The goal is not simply to remove permissions; it is to remove unnecessary access without breaking legitimate behavior.

What goes into Model 2

- Static evidence: imports, reachable calls, data flow, configuration, and referenced resources.
- Successful runtime events: file categories, network destinations, secret names, database actions, subprocesses, and relevant system calls.
- Functional-test outcomes: whether the application's expected tasks still succeed under a candidate policy.
- Existing policy information: declared permissions, broad defaults, and resource scopes.

What the model should return

- A multi-label policy covering file, network, secret, database, cloud-service, and process permissions.
- The resource scope for each grant, such as one folder or one API domain rather than all resources.
- A confidence score and evidence showing why the permission was recommended.
- An approval request for sensitive or uncertain permissions.

How Model 2 works

1. Use GraphCodeBERT as a static encoder for code-level resource references and data flow (Guo et al., 2021).
2. Run the app's normal functional tests inside a broad but isolated policy and record successful resource events.
3. Encode the event sequence with a Transformer. This branch learns which access commonly occurs together during legitimate operations.
4. Fuse the static and behavioral representations with cross-attention.
5. Predict the minimum permission labels and their scopes. Abstain when evidence is insufficient or the requested access is sensitive.
6. Validate the candidate policy by rerunning functional and attack tests before deployment.

Illustrative policy for the receipt app

```
{
  "file.read":  ["/app/uploads/*"],
  "file.write": ["/app/reports/*"],
  "network":    ["https://ocr.example.test"],
  "secret.read": ["OCR_API_KEY"],
  "database":   ["expenses:insert", "expenses:read"],
  "process.spawn": false
}
```

This policy permits only the receipt application's expected behavior. It does not allow the entire filesystem, arbitrary internet access, every secret, database administration, or subprocess creation. The domain is intentionally fictional and the policy is a simplified teaching example.

Simplified learning logic

```
static_vector = code_encoder(source_code, data_flow)
behavior_vector = event_transformer(successful_runtime_events)

combined = cross_attention(static_vector, behavior_vector)
permissions, confidence = permission_head(combined)

if confidence < threshold or contains_sensitive_grant(permissions):
    request_human_approval(permissions, evidence)
```

Creating trustworthy permission labels

1. Start with a broad policy inside an isolated test environment.
2. Remove one permission or narrow one scope at a time.
3. Rerun the complete functional tests.
4. Retain the permission only if removing it breaks an expected operation.
5. Manually review sensitive grants and incomplete test coverage.

This procedure adapts the drop-all/add-back idea in KubeCap (Liu et al., 2026) and the separation between task sufficiency and policy tightness in AuthBench (Yan et al., 2026). Functional tests must be sufficiently complete; otherwise, a rarely used but legitimate permission could be removed by mistake.

Data and evaluation for Model 2

- No reviewed public dataset exactly matches fine-grained permissions for the planned micro-app platform, so the main training labels must come from the team's applications and functional tests.
- Use Confine as a published static syscall-policy baseline (Rostamipoor et al., 2023).
- Use IAM Policy Autopilot as an auditable code-to-IAM comparison where AWS SDK calls are present (Filiari et al., 2026).
- Use ALPS as evidence for combining static discovery with runtime enforcement in serverless applications (Shin et al., 2026).
- Report multi-label F1, Jaccard similarity, overgrant rate, false-denial rate, unauthorized-action blocking, approval rate, and policy-generation time.

4. How the Models Work Together

Model 1 and Model 2 solve different but connected problems. Model 1 decides whether the application is safe enough to continue. Model 2 decides the narrowest access under which an accepted application can operate. Their outputs should be combined as evidence for deployment, not merged into one unexplained score.

Decision example

- Low Model 1 risk + narrow Model 2 policy: deploy after ordinary checks.
- Medium Model 1 risk + narrow policy: request a security review before deployment.
- Low risk + unexpectedly broad permissions: request approval or investigate missing functional-test coverage.
- High risk: block deployment even if the inferred permissions are narrow.

What is learned and what remains deterministic

- Deterministic checks: SBOM creation, exact OSV version matches, secret detection, policy enforcement, quotas, and isolation.
- Learned Model 1 components: code representation, graph relationships, vulnerability/category prediction, and risk calibration.
- Learned Model 2 components: static-behavior fusion, multi-label permission prediction, scope recommendation, and uncertainty.
- Human decision: approval of sensitive access and review of uncertain or high-impact cases.

How the literature strengthens the project

- SecBench.js provides executable JavaScript vulnerabilities and evaluation oracles (Bhuiyan et al., 2023).
- GraphCodeBERT provides the data-flow-aware encoder shared by both models (Guo et al., 2021).
- PrimeVul provides the leakage-resistant benchmark and split strategy (Ding et al., 2025).
- LLMxCPG provides security-focused graph slicing for Model 1 context (Lekssays et al., 2025).
- Confine provides the main published policy-generation baseline for Model 2 (Rostamipoor et al., 2023).

- IAM Policy Autopilot contributes explainable code-to-permission mapping (Filieri et al., 2026).
- ALPS contributes the link between static extraction and runtime policy verification (Shin et al., 2026).
- AuthBench and KubeCap contribute policy sufficiency, tightness, and minimization ideas, but both are recent preprints and should be identified as such (Liu et al., 2026; Yan et al., 2026).

What is new in our project

The safest novelty claim is the evaluated integration, not that each component is completely new. The project combines repository-level analysis of agent-generated micro-apps, code and dependency graph learning, static-dynamic permission inference, explicit approval, sandbox enforcement, runtime monitoring, rollback, and resource-aware scheduling. It also proposes MicroAppSec-100 as an original end-to-end benchmark with prompt/model provenance, functional tests, security labels, permission ground truth, and deployment measurements.

5. Evaluation: What Each Metric Means

- Deployment success: the percentage of legitimate apps that build, start, and pass their functional tests.
- Exploit-blocking recall: blocked seeded exploits divided by all seeded exploits. This is the key integrated security measure for Model 1.
- False-denial rate: legitimate required actions denied divided by all legitimate required actions. This is essential for Model 2 usability.
- Isolation escapes: successful attempts to cross the sandbox or tenant boundary. This is a platform result, not only a Model 1 or Model 2 result.
- p95 cold start: the startup time below which 95% of cold starts finish. This primarily evaluates Model 4.
- Cost per app-hour: the infrastructure cost to keep one application available for one hour. This primarily evaluates Model 4 and platform efficiency.
- Recovery time: the time from detection to termination and restoration of a safe version.

Recommended success criterion

A defensible project target is to block at least 90% of seeded exploits while keeping false denials at or below 5% on held-out MicroAppSec-100 application families. It must be written as a target until the team completes the experiment. Paper results must be introduced as results reported by those studies, not as results achieved by this project.

6. Practical Build Order

1. Create a 20-to-30-app pilot and define functional tests, SBOMs, vulnerability labels, and permission labels.
2. Implement rules-only baselines: OSV/scanner risk checks for Model 1 and Confine or SDK-call mapping for Model 2.
3. Train the GraphCodeBERT code branch and add the dependency graph for Model 1.
4. Collect successful runtime events and create reviewed minimum policies.
5. Train the event Transformer and static-dynamic fusion for Model 2.
6. Evaluate each model separately, then test the integrated deployment workflow.
7. Expand to 100 apps only after the pilot labels and test harness are reliable.

7. Papers and Direct Links

- Bhuiyan, M. H. M., Parthasarathy, A. S., Vasilakis, N., Pradel, M., & Staicu, C.-A. (2023). SecBench.js: An executable security benchmark suite for server-side JavaScript. [Direct paper PDF](#)
- Ding, Y., Fu, Y., Ibrahim, O., Sitawarin, C., Chen, X., Alomair, B., Wagner, D., Ray, B., & Chen, Y. (2025). Vulnerability detection with code language models: How far are we? [Open paper](#)
- Filieri, A., Kennedy, L., Luo, K., Luttrell, M., Palacios, A., Panda, A., Rungta, N., Schlaipfer, M., Singh, K. J., Tanuku, A., Winans, N., Yin, D., & Zhang, W. (2026). IAM Policy Autopilot: Static analysis for policy generation from application code. [DOI and paper record](#)
- Guo, D., et al. (2021). GraphCodeBERT: Pre-training code representations with data flow. [Open paper](#)
- Lekssays, A., Mouhcine, H., Tran, K., Yu, T., & Khalil, I. (2025). LLMxCPG: Context-aware vulnerability detection through code property graph-guided large language models. [Direct paper PDF](#)
- Lian, K., et al. (2025). A.S.E.: A repository-level benchmark for evaluating security in AI-generated code [Preprint]. [Open paper](#)
- Liu, Y., Zhou, Y., Liu, W., Jia, Y., & Liu, Z. (2026). KubeCap: A framework for capability minimization in Kubernetes via static analysis and LLM-assisted rule inference [Preprint]. [Open paper](#)
- Rostamipoor, M., Ghavamnia, S., & Polychronakis, M. (2023). Confine: Fine-grained system call filtering for container attack surface reduction. Computers & Security, 132, 103325. [DOI](#)
- Shin, C., Kim, B., & Lee, S. (2026). ALPS: Automated least-privilege enforcement for securing serverless functions. [Open paper](#)
- Yan, Z., et al. (2026). Do coding agents understand least-privilege authorization? [Preprint]. [Open paper](#)

Datasets and Implementations

- SecBench.js dataset and code: [direct repository](#).
- PrimeVul data and experiments: [direct repository](#).
- GraphCodeBERT implementation: [direct repository](#).
- Confine implementation: [direct repository](#).
- Open Source Vulnerabilities database: [direct database](#).